

Competitive Programming

Section 40 Editorial: Data Structures Mixed Contest

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
2	A. Hotel Queries	4
2.1	Restatement	4
2.2	Algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	5
3	B. List Removals	7
3.1	Restatement	7
3.2	Algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	8
4	C. Mo Distinct Queries	10
4.1	Restatement	10
4.2	Algorithm	10
4.3	Correctness proof	10
4.4	Complexity	10
4.5	Full C++ solution	10
4.6	Full Python solution	11
5	Testing discussion	13
6	Detailed upsolve workshop	14
6.1	A. Hotel Queries	14
6.2	B. List Removals	14
6.3	C. Mo Distinct Queries	15
7	What to carry forward	16

1 Editorial overview

Problem	Main idea
A. Hotel Queries	See full algorithm and code below.
B. List Removals	See full algorithm and code below.
C. Mo Distinct Queries	See full algorithm and code below.

2 A. Hotel Queries

2.1 Restatement

For each group size, place it in the first hotel with enough remaining capacity, subtract that size, and print the hotel index or 0.

2.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

2.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. \square

2.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n, q;
if (!(cin >> n >> q)) return 0;
int size = 1;
while (size < n) size <= 1;
vector<long long> seg(2 * size, 0);
for (int i = 0; i < n; i++) cin >> seg[size + i];
for (int i = size - 1; i >= 1; i--) seg[i] = max(seg[2 * i], seg[2 * i + 1]);
for (int qi = 0; qi < q; qi++) {
    long long x;
    cin >> x;
    int ans = 0;
    if (seg[1] >= x) {
        int v = 1;
        while (v < size) {
            if (seg[2 * v] >= x) v *= 2;
            else v = 2 * v + 1;
        }
        ans = v - size + 1;
        seg[v] -= x;
        for (v >= 1; v; v >= 1) seg[v] = max(seg[2 * v], seg[2 * v + 1]);
    }
    if (qi) cout << ' ';
    cout << ans;
}
cout << '\n';
return 0;
}

```

2.6 Full Python solution

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    req = data[2 + n:2 + n + q]
    size = 1
    while size < n:
        size *= 2
    seg = [0] * (2 * size)
    for i, x in enumerate(a):
        seg[size + i] = x
    for i in range(size - 1, 0, -1):
        seg[i] = max(seg[2 * i], seg[2 * i + 1])
    out = []
    for x in req:

```

```

if seg[1] < x:
    out.append("0")
    continue
v = 1
while v < size:
    if seg[2 * v] >= x:
        v *= 2
    else:
        v = 2 * v + 1
pos = v - size
seg[v] -= x
v //= 2
while v:
    seg[v] = max(seg[2 * v], seg[2 * v + 1])
    v //= 2
out.append(str(pos + 1))
print(" ".join(out))

if __name__ == "__main__":
    main()

```

3 B. List Removals

3.1 Restatement

Repeatedly remove and print the k -th currently alive element.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Fenwick {
    int n;
    vector<int> bit;
    Fenwick(int n) : n(n), bit(n + 1, 0) {}
};
```

```

void add(int i, int delta) {
    for (; i <= n; i += i & -i) bit[i] += delta;
}

int kth(int k) const {
    int pos = 0;
    int step = 1;
    while ((step << 1) <= n) step <<= 1;
    for (; step; step >>= 1) {
        int nxt = pos + step;
        if (nxt <= n && bit[nxt] < k) {
            pos = nxt;
            k -= bit[nxt];
        }
    }
    return pos + 1;
}

};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    Fenwick bit(n);
    for (int i = 1; i <= n; i++) bit.add(i, 1);
    for (int t = 0; t < n; t++) {
        int k;
        cin >> k;
        int idx = bit.kth(k);
        if (t) cout << ' ';
        cout << a[idx];
        bit.add(idx, -1);
    }
    cout << '\n';
    return 0;
}

```

3.6 Full Python solution

```

import sys

class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)
    def add(self, i, delta):

```

```

        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
def kth(self, k):
    pos = 0
    bit = 1 << self.n.bit_length()
    while bit:
        nxt = pos + bit
        if nxt <= self.n and self.bit[nxt] < k:
            pos = nxt
            k -= self.bit[nxt]
        bit //= 2
    return pos + 1

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:1 + n]
    positions = data[1 + n:1 + 2 * n]
    bit = Fenwick(n)
    for i in range(1, n + 1):
        bit.add(i, 1)
    out = []
    for k in positions:
        idx = bit.kth(k)
        out.append(str(a[idx - 1]))
        bit.add(idx, -1)
    print(" ".join(out))

if __name__ == "__main__":
    main()

```

4 C. Mo Distinct Queries

4.1 Restatement

For each offline range query, print the number of distinct values in the range.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Query {
    int l, r, id;
};
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<int> a(n), vals;
    for (int &x : a) {
        cin >> x;
        vals.push_back(x);
    }
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    for (int &x : a) x = int(lower_bound(vals.begin(), vals.end(), x) - vals.begin());
    vector<Query> queries(q);
    for (int i = 0; i < q; i++) {
        cin >> queries[i].l >> queries[i].r;
        --queries[i].l; --queries[i].r;
        queries[i].id = i;
    }
    int block = max(1, (int)sqrt(n));
    sort(queries.begin(), queries.end(), [&](const Query& A, const Query& B) {
        int ba = A.l / block, bb = B.l / block;
        if (ba != bb) return ba < bb;
        return (ba & 1) ? A.r > B.r : A.r < B.r;
    });
    vector<int> freq(vals.size(), 0), ans(q);
    int distinct = 0, l = 0, r = -1;
    auto add = [&](int pos) {
        if (freq[a[pos]]++ == 0) distinct++;
    };
    auto remove = [&](int pos) {
        if (--freq[a[pos]] == 0) distinct--;
    };
    for (auto qu : queries) {
        while (r < qu.r) add(++r);
        while (r > qu.r) remove(r--);
        while (l < qu.l) remove(l++);
        while (l > qu.l) add(--l);
        ans[qu.id] = distinct;
    }
    for (int x : ans) cout << x << '\n';
    return 0;
}

```

4.6 Full Python solution

```

import sys
from collections import defaultdict

```

```

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    idx = 2 + n
    block = max(1, int(n ** 0.5))
    queries = []
    for qi in range(q):
        l, r = data[idx] - 1, data[idx + 1] - 1
        idx += 2
        queries.append((l, r, qi))
    queries.sort(key=lambda x: (x[0] // block, x[1] if (x[0] // block) % 2 == 0 else -x[1]))
    freq = defaultdict(int)
    ans = [0] * q
    distinct = 0
    left, right = 0, -1
    for l, r, qi in queries:
        while right < r:
            right += 1
            freq[a[right]] += 1
            if freq[a[right]] == 1:
                distinct += 1
        while right > r:
            freq[a[right]] -= 1
            if freq[a[right]] == 0:
                distinct -= 1
            right -= 1
        while left < l:
            freq[a[left]] -= 1
            if freq[a[left]] == 0:
                distinct -= 1
            left += 1
        while left > l:
            left -= 1
            freq[a[left]] += 1
            if freq[a[left]] == 1:
                distinct += 1
        ans[qi] = distinct
    print("\n".join(map(str, ans)))

if __name__ == "__main__":
    main()

```

5 Testing discussion

6 Detailed upsolve workshop

6.1 A. Hotel Queries

Constraints and trap. n, q are up to about $2e5$. A linear scan for each group is too slow; a plain maximum alone finds feasibility but not the first index. Store the maximum remaining capacity in each segment-tree node.

Trace and pseudocode. With capacities $[3, 1, 4, 1, 5]$, a request of four descends root $\rightarrow$ left/right according to whether the left subtree maximum is at least four, reaches index three, and changes it to zero.

```
if root.max < x: output 0
otherwise descend: go left if left.max >= x, else right
at leaf i: subtract x; recompute max values on the path to root; output i
```

Proof and complexity. Every node is the maximum of exactly its leaves. A feasible left child contains an earlier valid hotel; an infeasible left child contains none, so the descent returns the first valid leaf. Each query changes and rebuilds one root-to-leaf path: $O(\log n)$ time, $O(n)$ memory. Use long long capacities. Test no feasible hotel, the first/last feasible hotel, and repeated requests to one hotel.

6.2 B. List Removals

Constraints and trap. Removing an element from an array/vector shifts all following values and becomes quadratic. Instead, original positions are fixed; a Fenwick tree stores one for alive positions and zero for removed positions.

Trace and pseudocode. For values $[10, 20, 30, 40, 50]$, rank two resolves to original index two, outputs 20, then adds -1. With counts $[1, 0, 1, 1, 1]$, rank three resolves to original index four, outputting 40.

```
initialize BIT with one at every index
for rank k:
    i = first index whose BIT prefix sum >= k
    output value[i]; BIT.add(i, -1)
```

Proof and complexity. The BIT prefix sum is the number of alive original positions at or before its index. Therefore its first prefix reaching k is the k -th current item. Fenwick binary lifting gives $O(\log n)$ per removal and $O(n)$ memory. The search uses one-based indices; test rank one, last rank, and duplicate values. A small mutable-list simulation is the stress oracle.

6.3 C. Mo Distinct Queries

Constraints and trap. Static array, many offline range queries, and no updates signal Mo's algorithm. Recomputing a set for every range is quadratic in the worst case. Coordinate-compress values before using an array frequency.

Trace and pseudocode. For $[1,2,1,3,2,4]$, activate $[1,3]$ to obtain frequencies $1:2, 2:1$ and answer two. Move to $[2,5]$: remove the first one, add 3 and 2, and obtain three.

```
sort queries by left block, alternating right direction
move [curL,curR] one endpoint at a time
add(pos): if freq[a[pos]] changes 0->1, distinct++
remove(pos): if freq[a[pos]] changes 1->0, distinct--
```

Proof and complexity. The frequency invariant exactly represents the current window, so `distinct` is correct when a query is reached. Block order limits endpoint movement to $O((n+q)\sqrt{n})$ in the standard analysis; memory is $O(n+q)$. Convert endpoints once, use buffered I/O in Python, and test all equal values, all distinct values, singletons, and repeated ranges against `len(set(...))` on small random arrays.

7 What to carry forward

Choose a stored summary that answers the decision you actually need: subtree maximum for first-fit, alive prefix count for dynamic rank, and frequencies for an offline window. The invariant makes both the data structure and its proof mechanical.

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.